ExtMem: Enabling Application-Aware Memory Page Placement Policies and Mechanisms

Sepehr Jalalian, **Shaurya Patel**, Milad Rezaei Hajidehi, Margo Seltzer, and Alexandra (Sasha) Fedorova

USENIX ATC'24

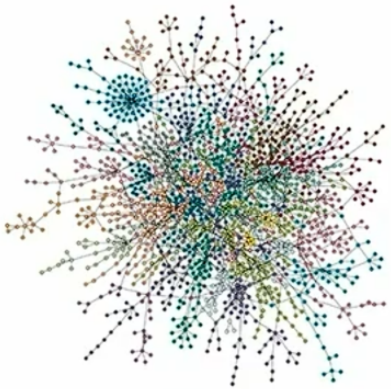
Memory is expensive

"In recent years, DRAM has become a critical bottleneck for scaling the WSCs" – Google, 2019

"DRAM device scaling has slowed down and it accounts for over 30% of datacenter cost" – Meta, 2022

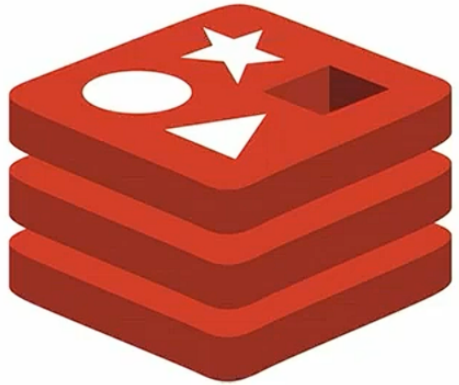
Applications are memory hungry

Diverse applications exhibit diverse memory access patterns



Graph Processing

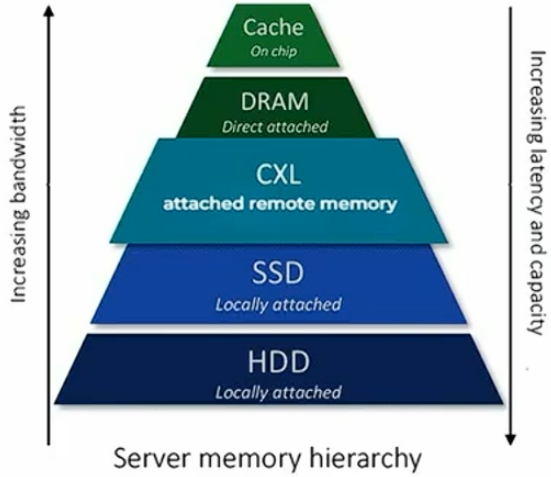
Machine Learning



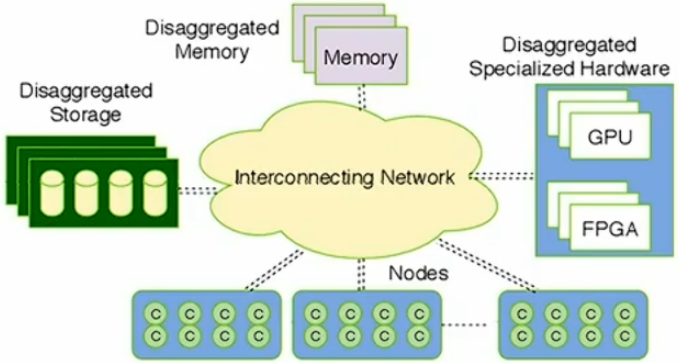
Databases

System setups are heterogenous

Memory Tiering



Disaggregation



Accelerators

Memory management policies need to be tailored to specific applications and hardware

Tiering image source - Rambus

Overarching Problem

- Current kernel memory management policies are insufficient
- Large overhead of existing userspace upcall (userfaultfd) mechanisms
- Rapidly changing environment there is a need to support new scenarios

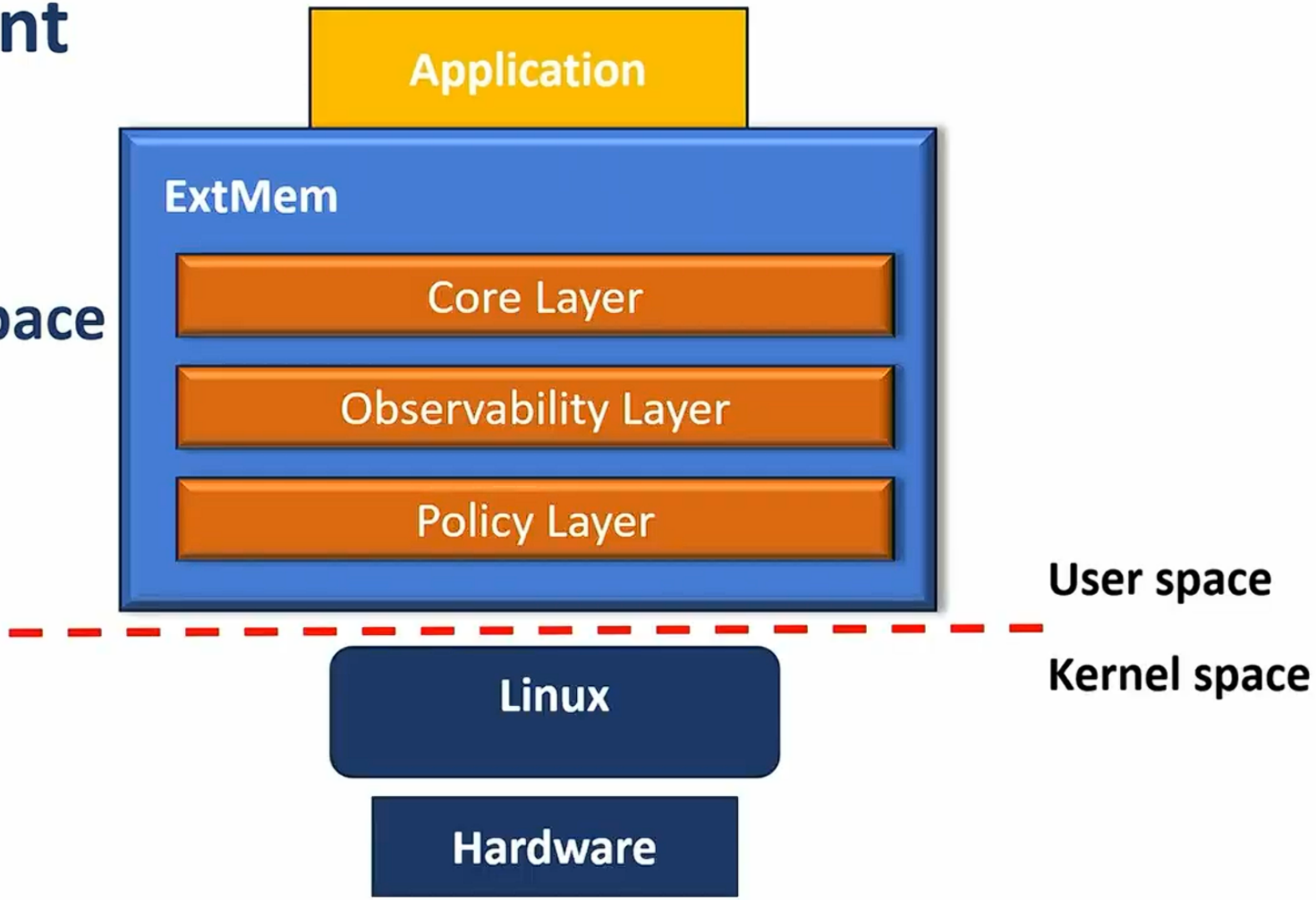
  but development inside kernel is hard

We need a solution that makes it **easy to develop new policies** for different systems **while maintaining performance** equivalent to the kernel!

SYSTOPIA

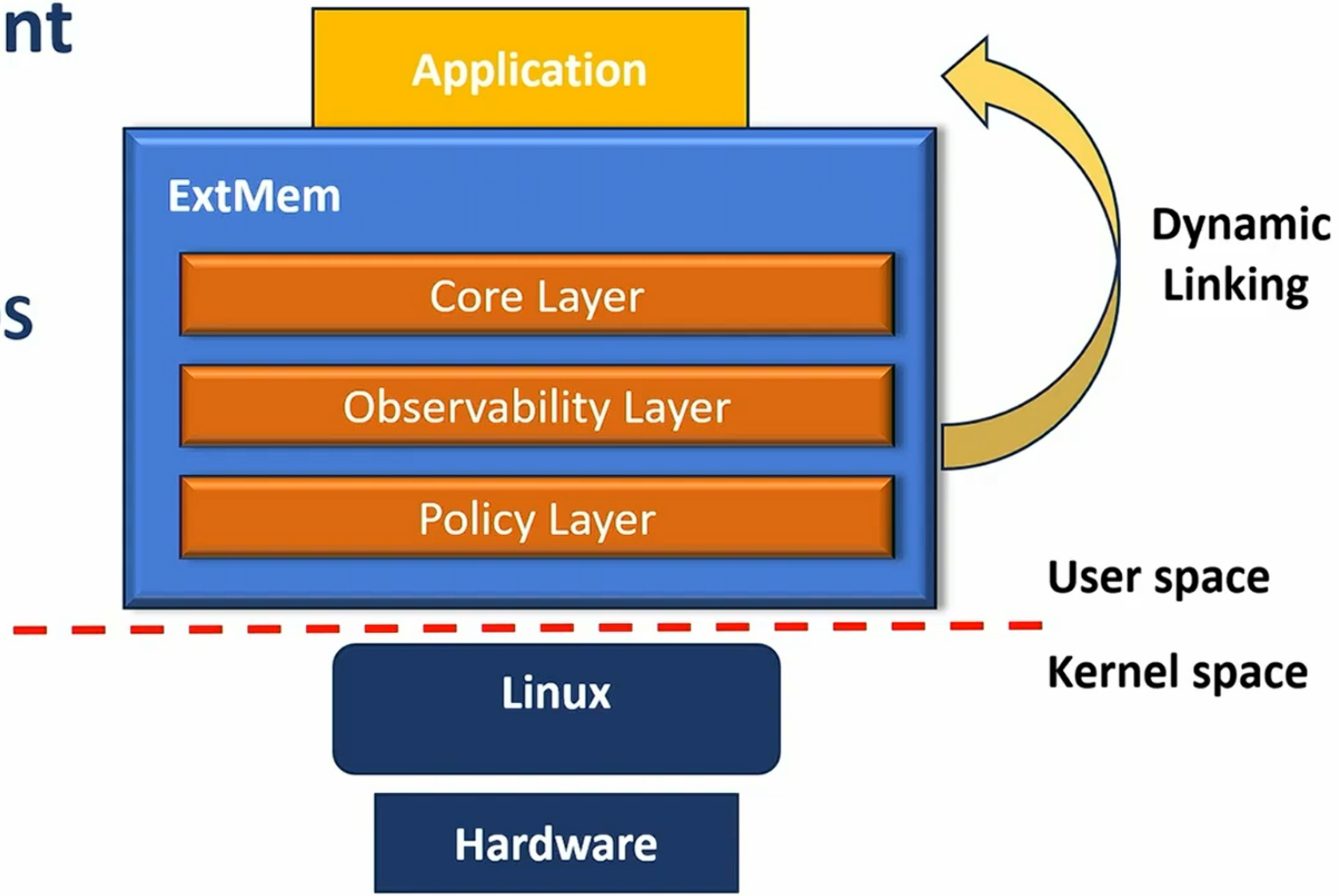
ExtMem: a framework for application-specific memory management

Runs in Userspace

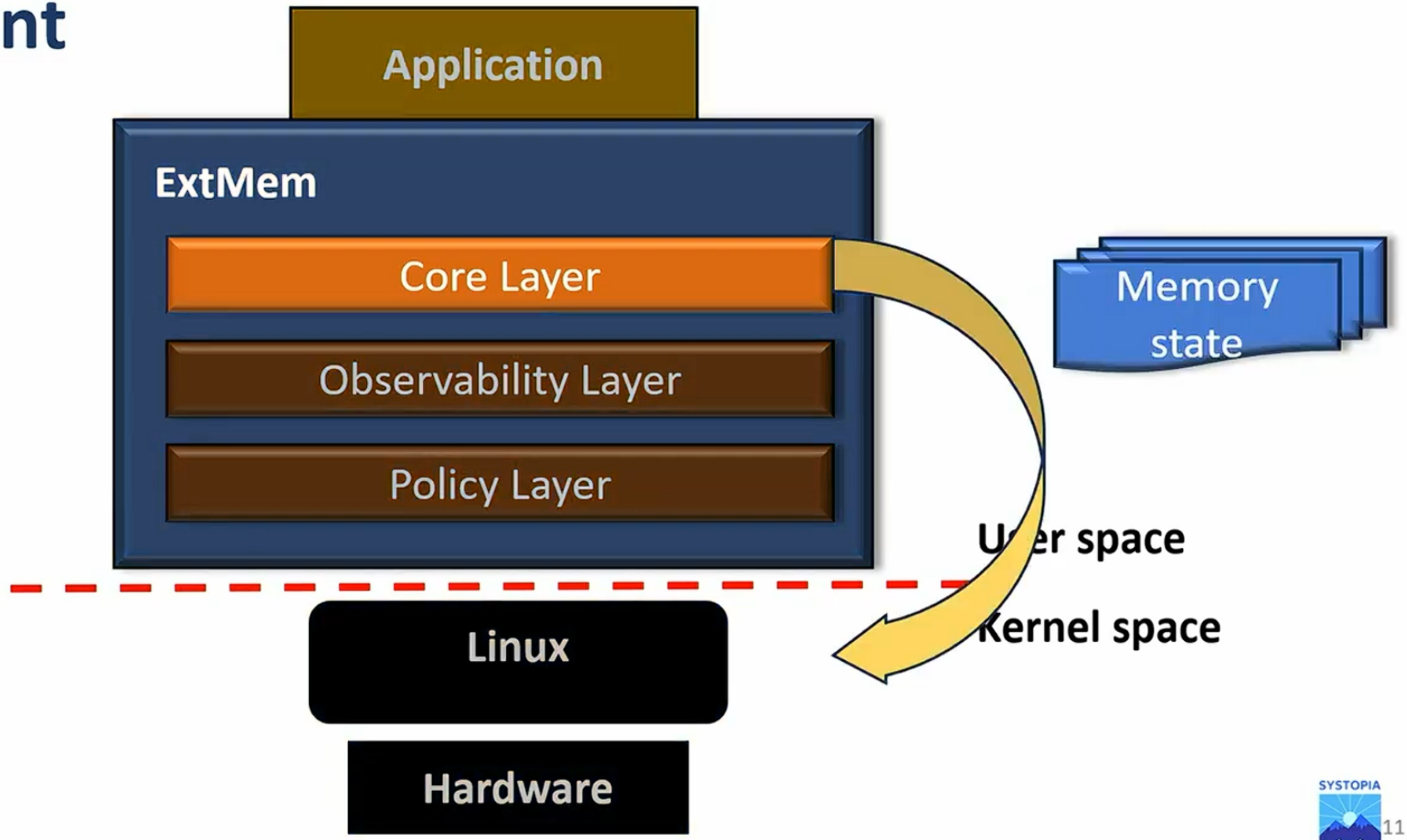


ExtMem: a framework for application-specific memory management

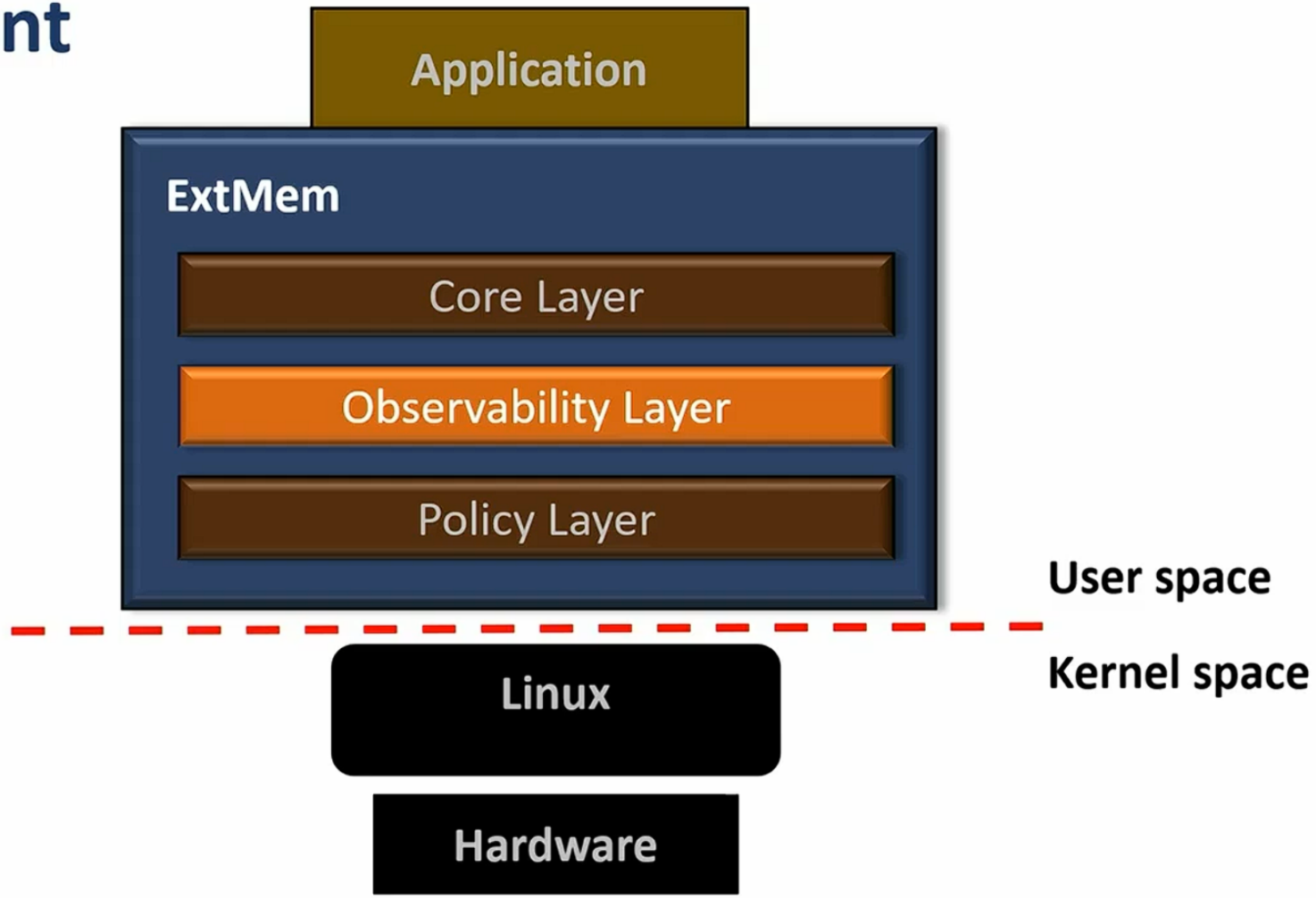
Similar to libOS



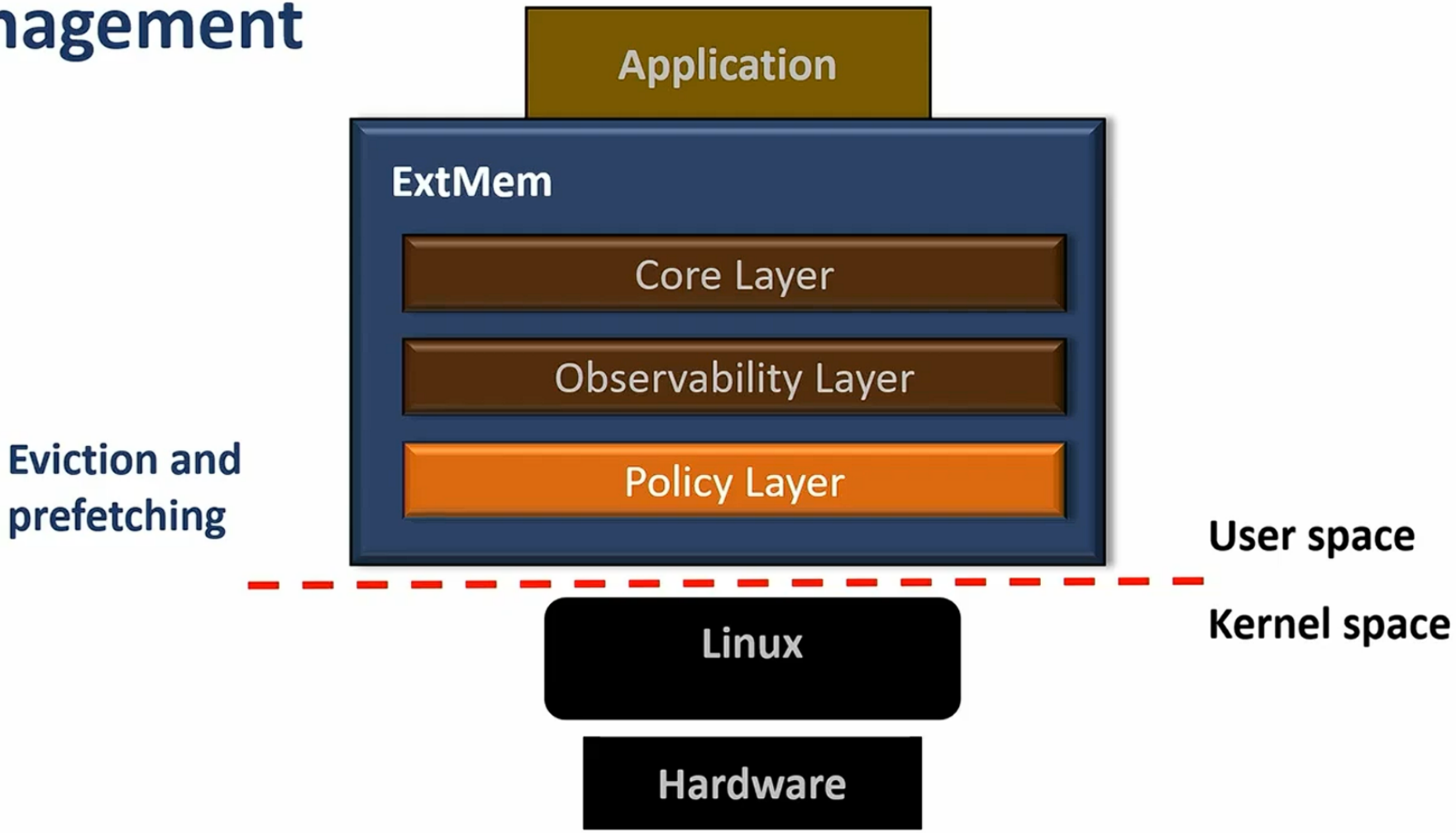
ExtMem: a framework for application-specific memory management



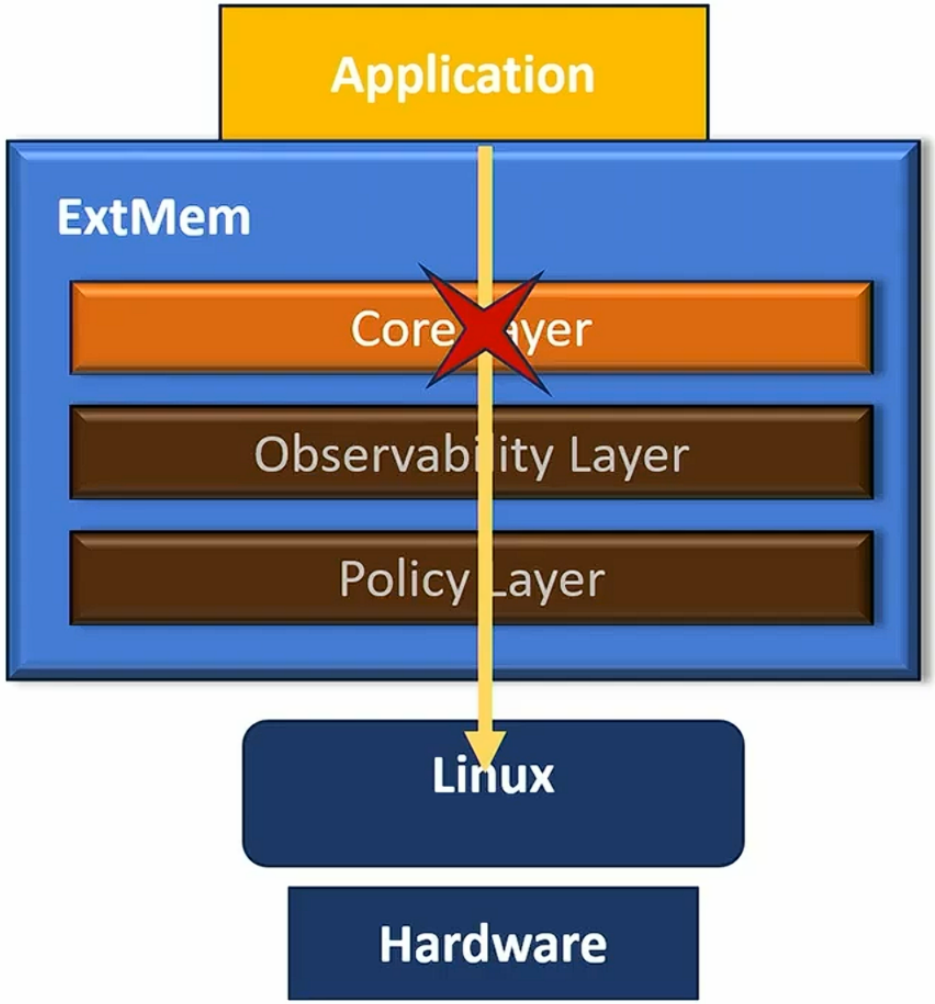
ExtMem: a framework for application-specific memory management



ExtMem: a framework for application-specific memory management



Core layer



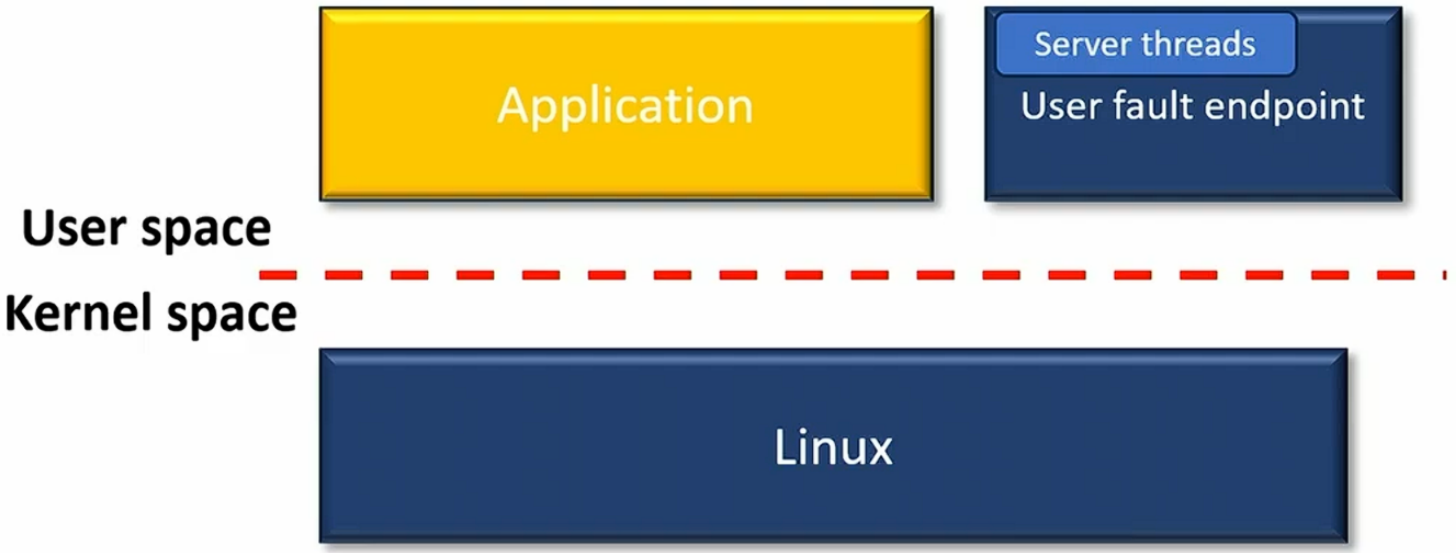
Transparent to Application code

libc patching

Memory system calls:
mmap
munmap
madvise
…

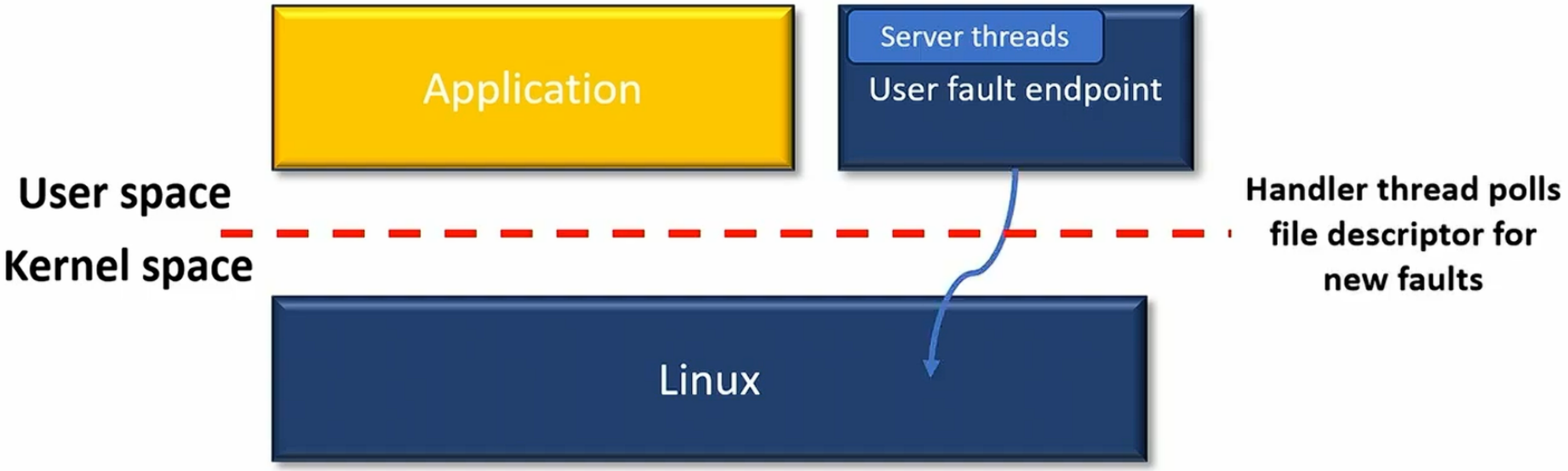
State of the art doesn't scale

Prior work based on userfaultfd doesn't scale due to expensive IPC and serialization



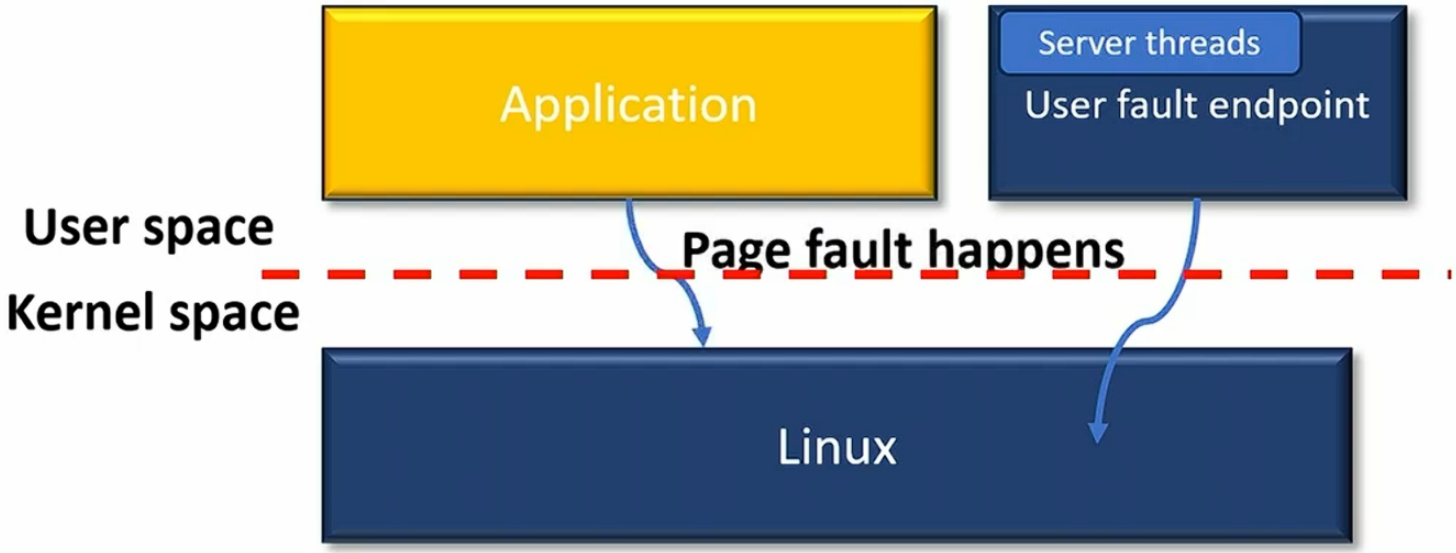
State of the art doesn't scale

Prior work based on userfaultfd doesn't scale due to expensive IPC and serialization



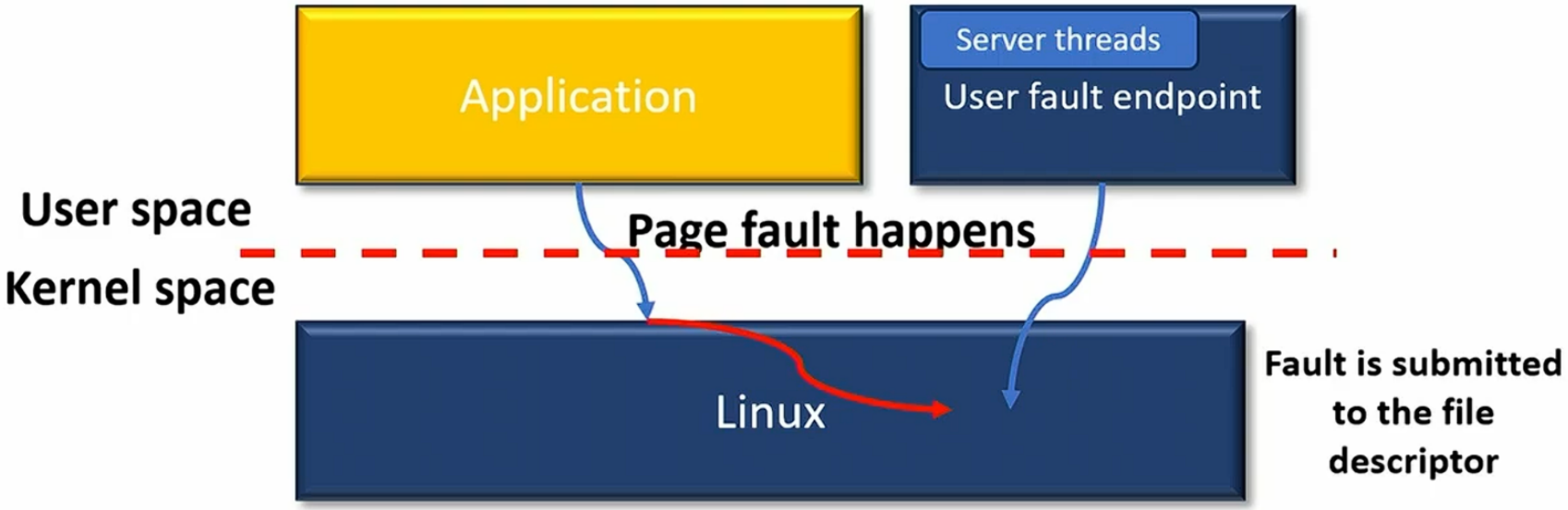
State of the art doesn't scale

Prior work based on userfaultfd doesn't scale due to expensive IPC and serialization



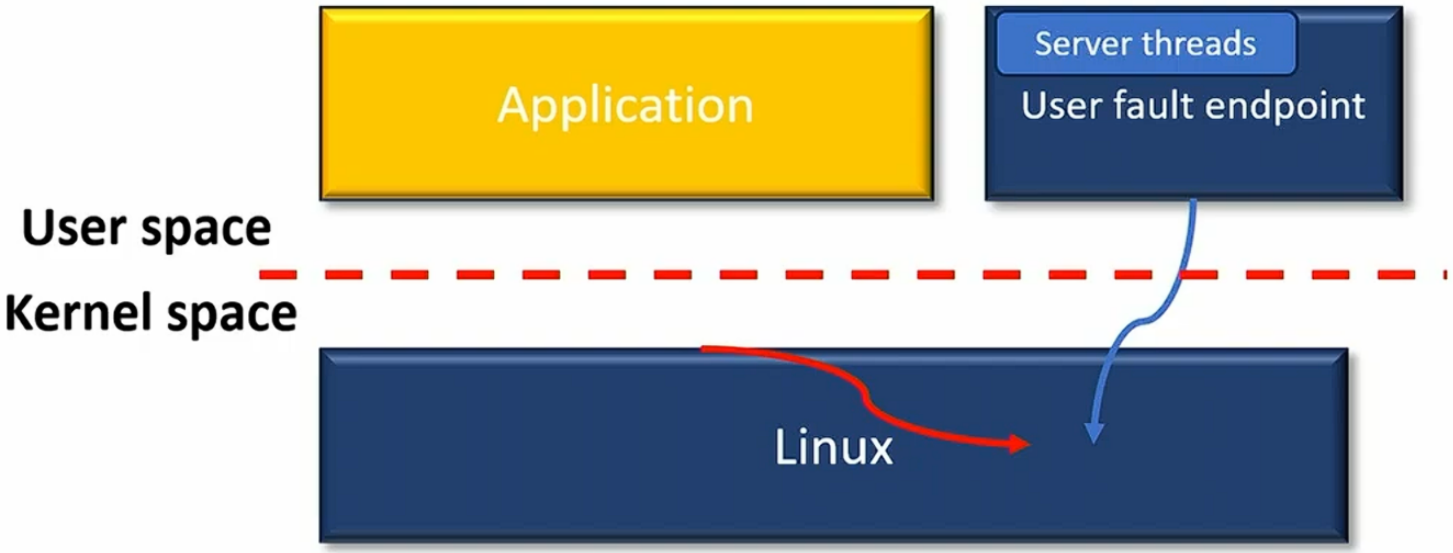
State of the art doesn't scale

Prior work based on userfaultfd doesn't scale due to expensive IPC and serialization



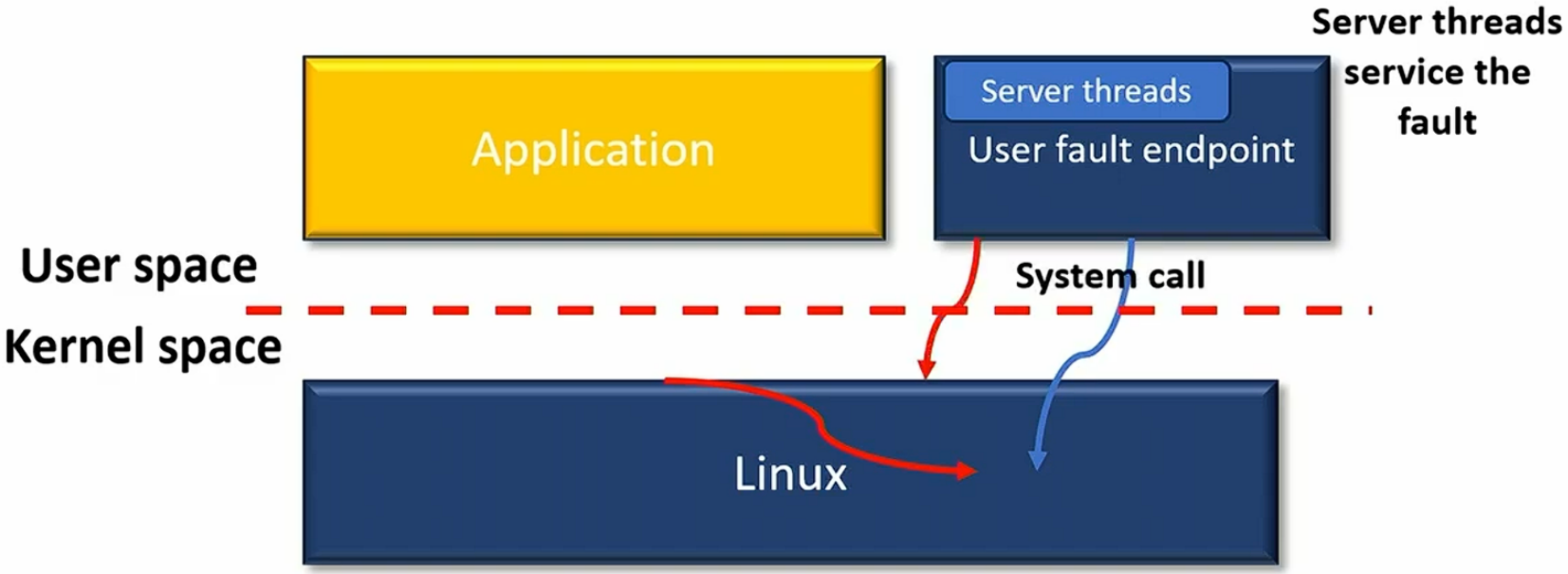
State of the art doesn't scale

Prior work based on userfaultfd doesn't scale due to expensive IPC and serialization



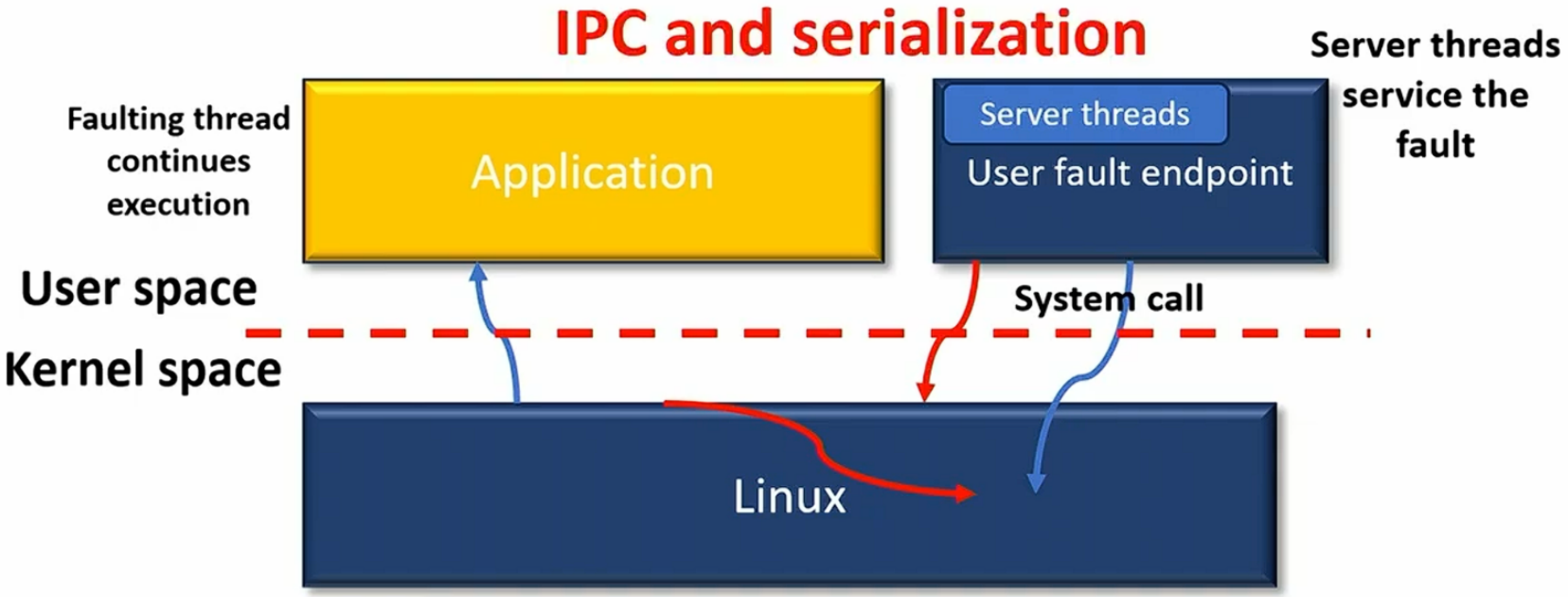
State of the art doesn't scale

Prior work based on userfaultfd doesn't scale due to expensive IPC and serialization

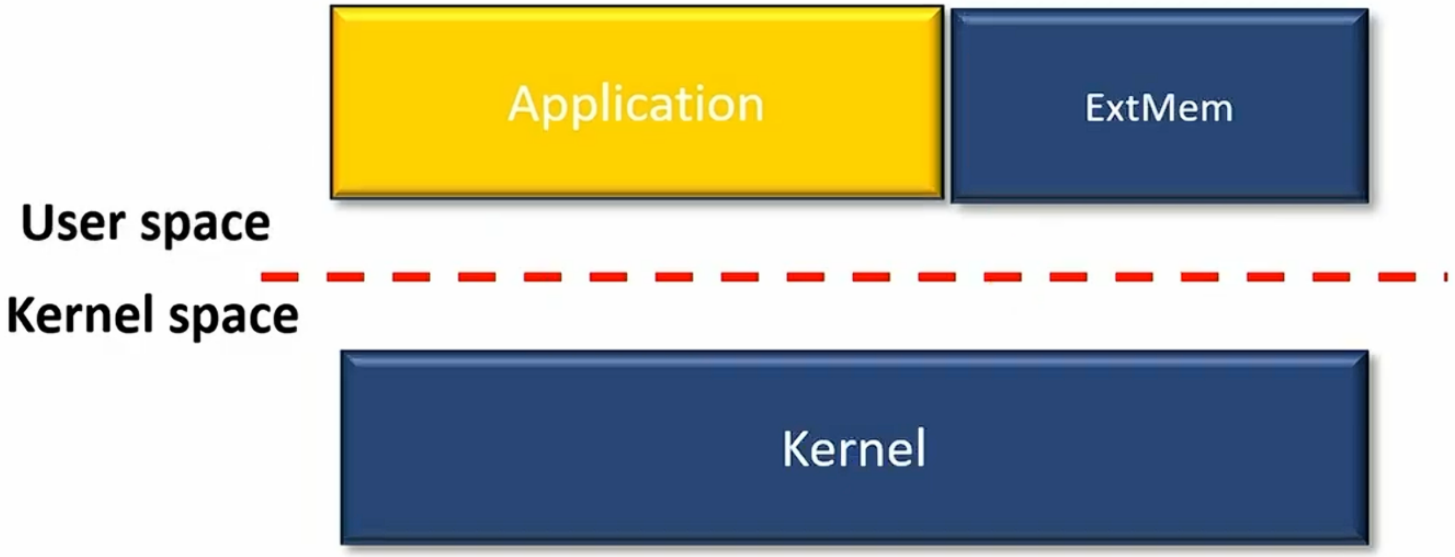


State of the art doesn't scale

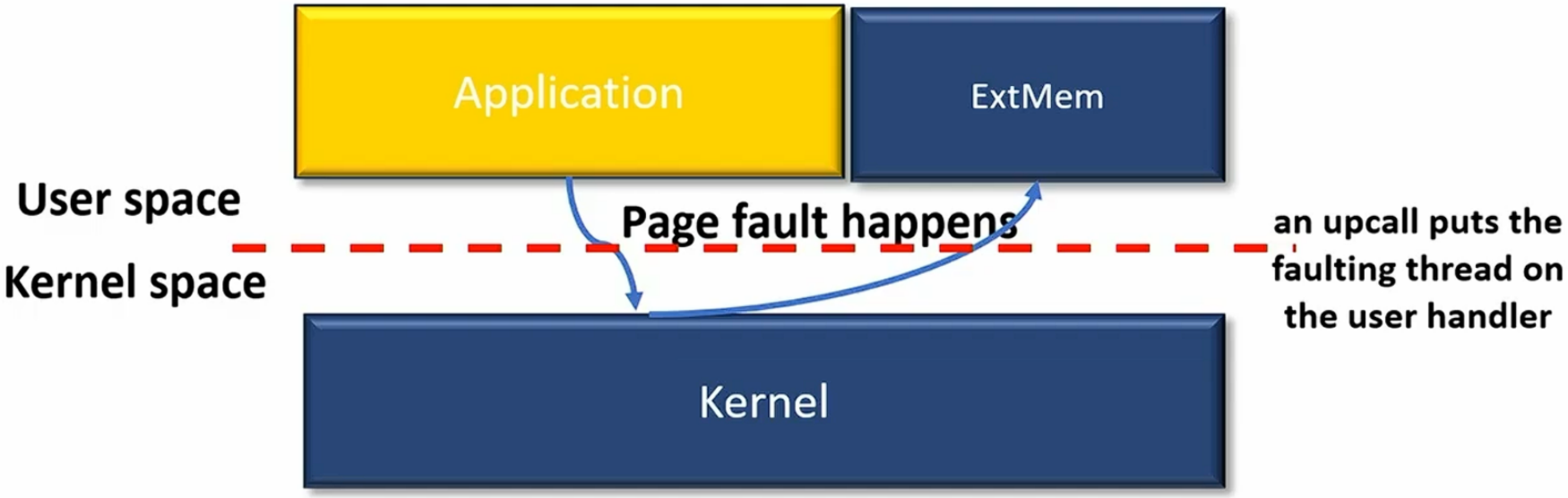
Prior work based on userfaultfd doesn't scale due to expensive IPC and serialization



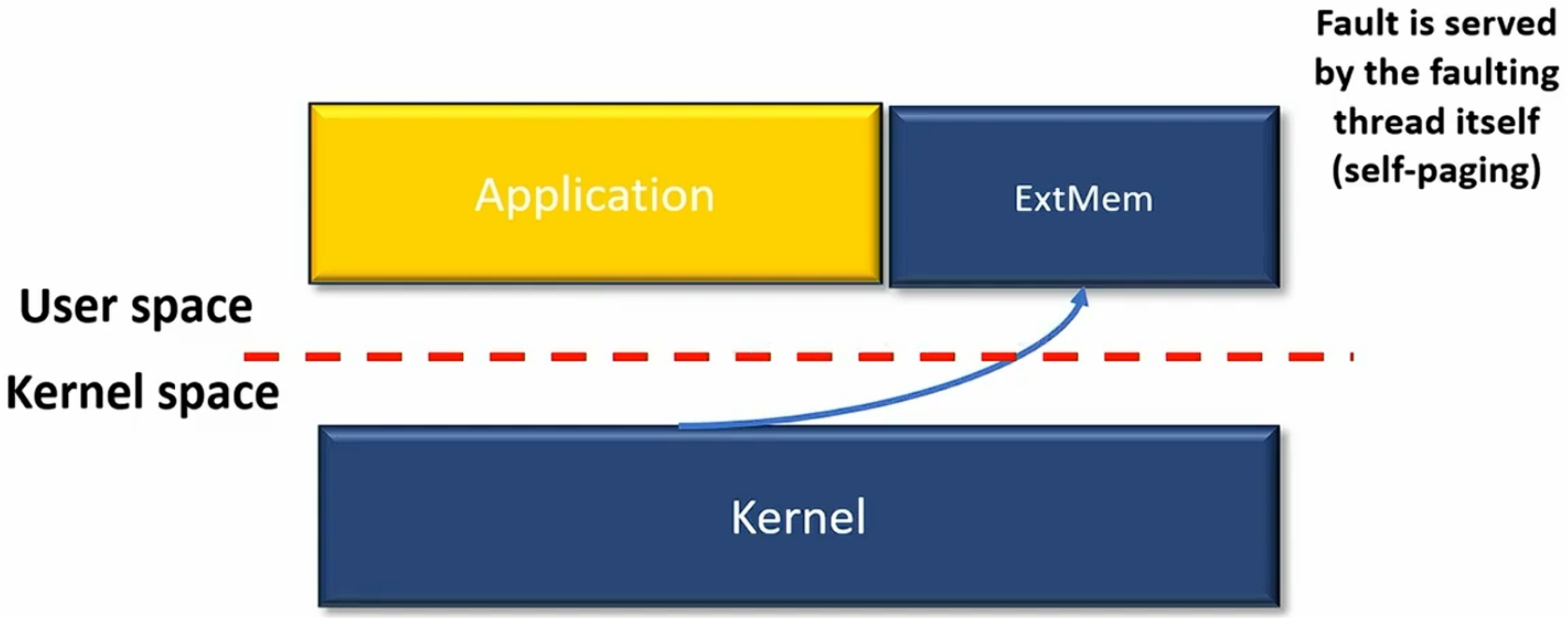
ExtMem scalable design



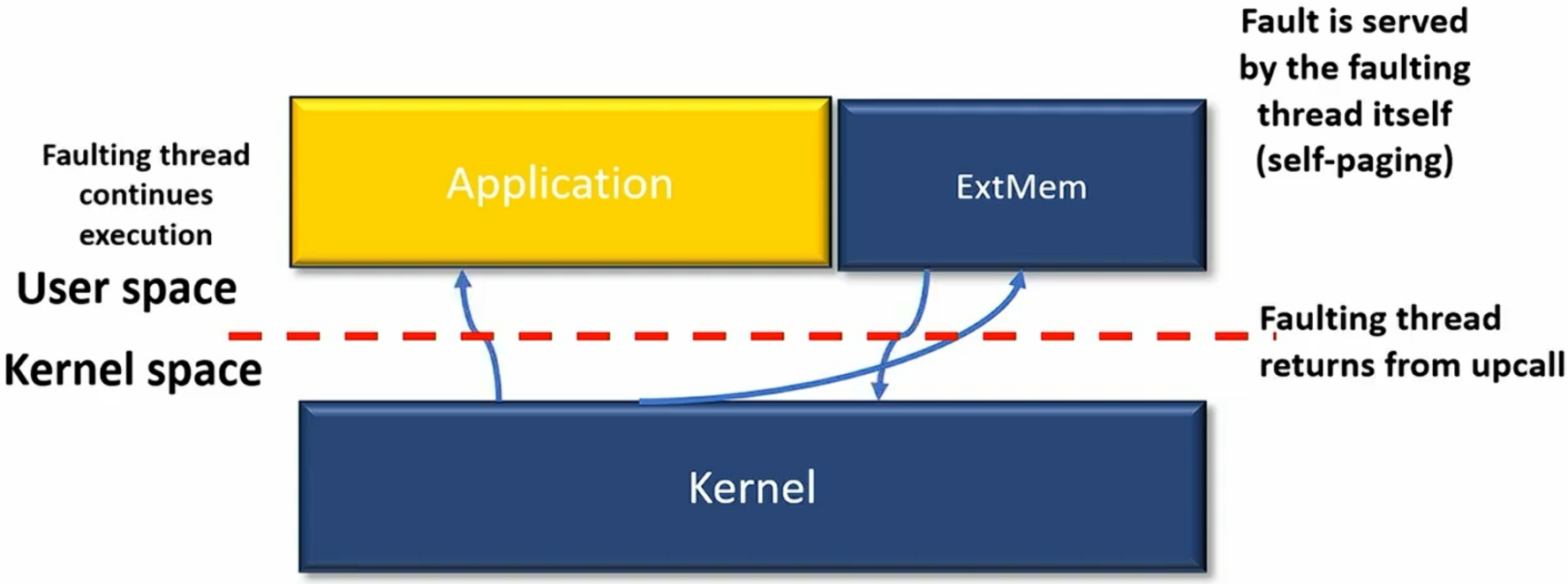
ExtMem scalable design



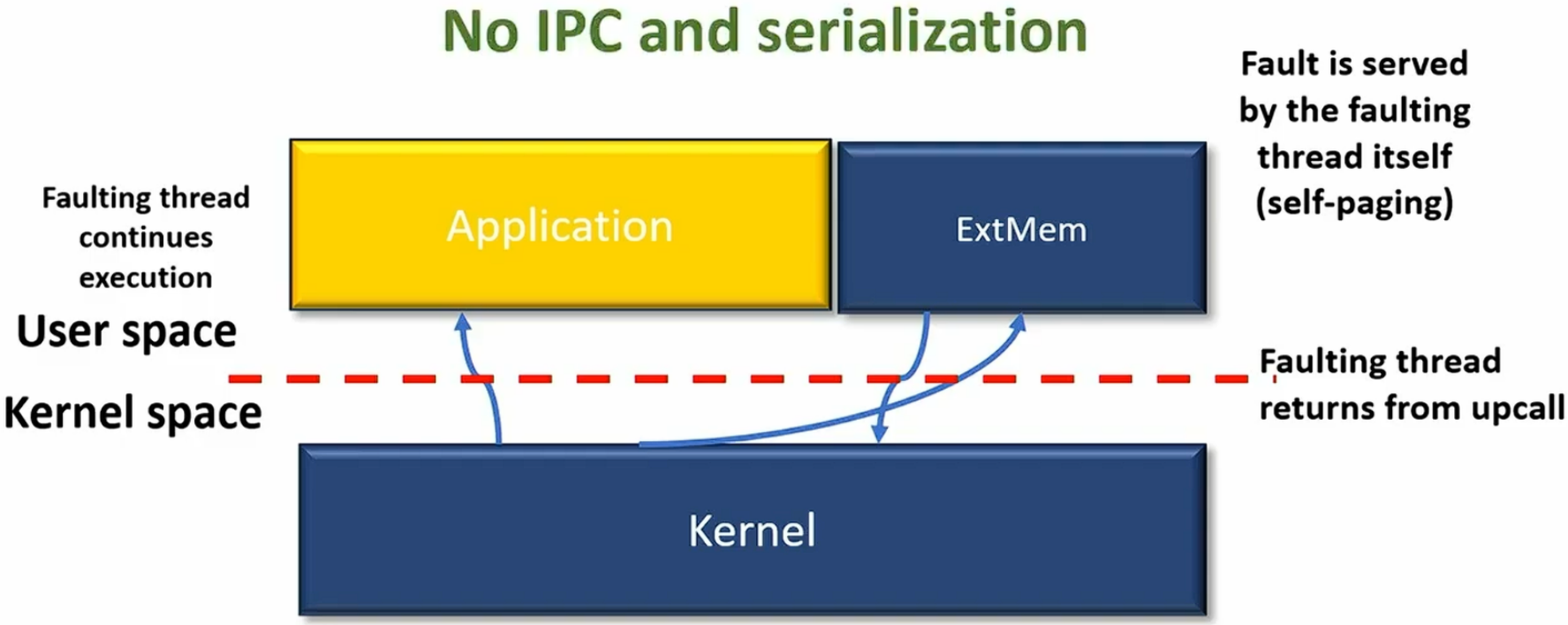
ExtMem scalable design



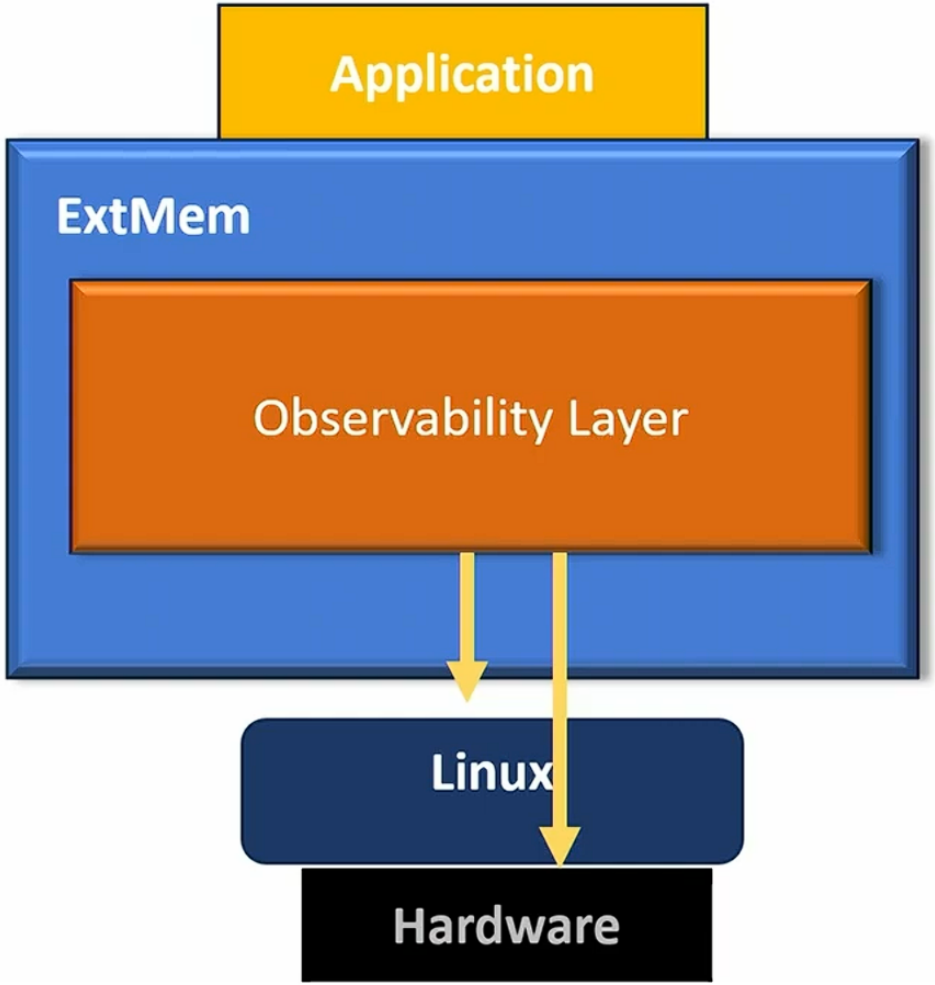
ExtMem scalable design



ExtMem scalable design



Observability layer



Telemetry:

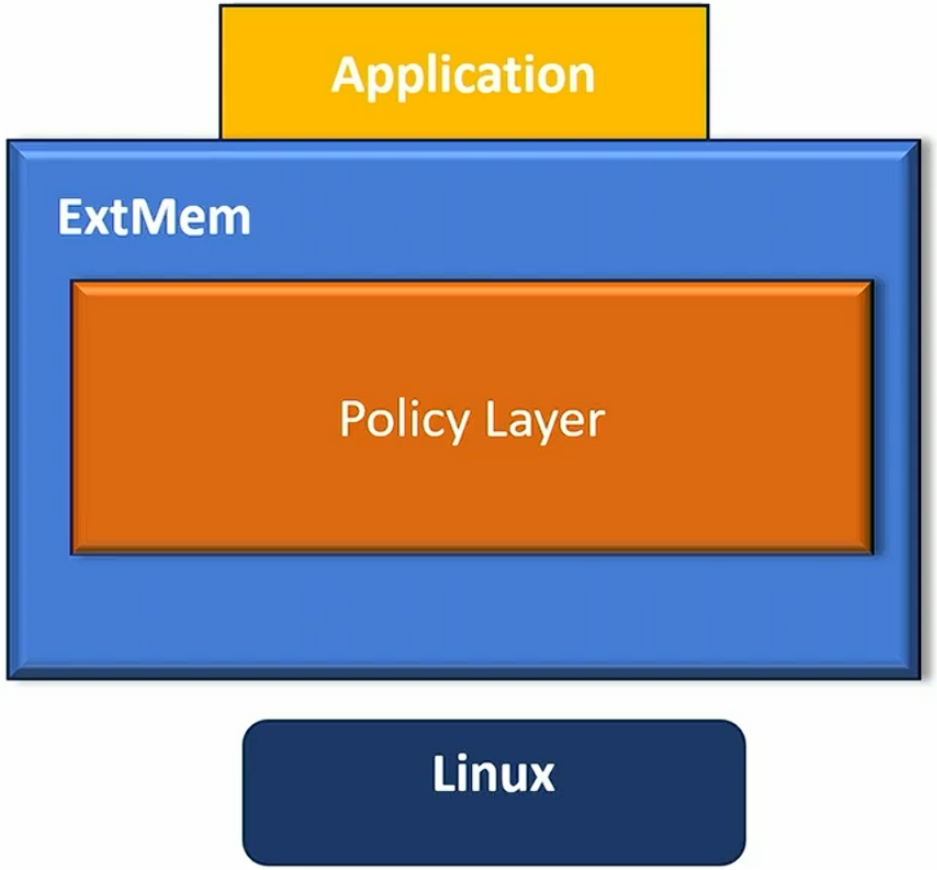
Page fault addresses

Accessed and dirty bits

Hardware counters

…

Policy layer



Example API:
Swap_pages(…)
Swap_async(…)
Try_prefetch(…)

Policies:
2QLRU – 300 loc vs Linux – ~500 loc
Sequential prefetcher – 200 loc

Evaluation

Q1 Is ExtMem's upcall method better than userfaultfd?

Q2 Is the overhead of ExtMem reasonable?

Q3 How do ExtMem's policies compare to Linux?

Q4 Can we improve application performance using ExtMem?

System Setup:

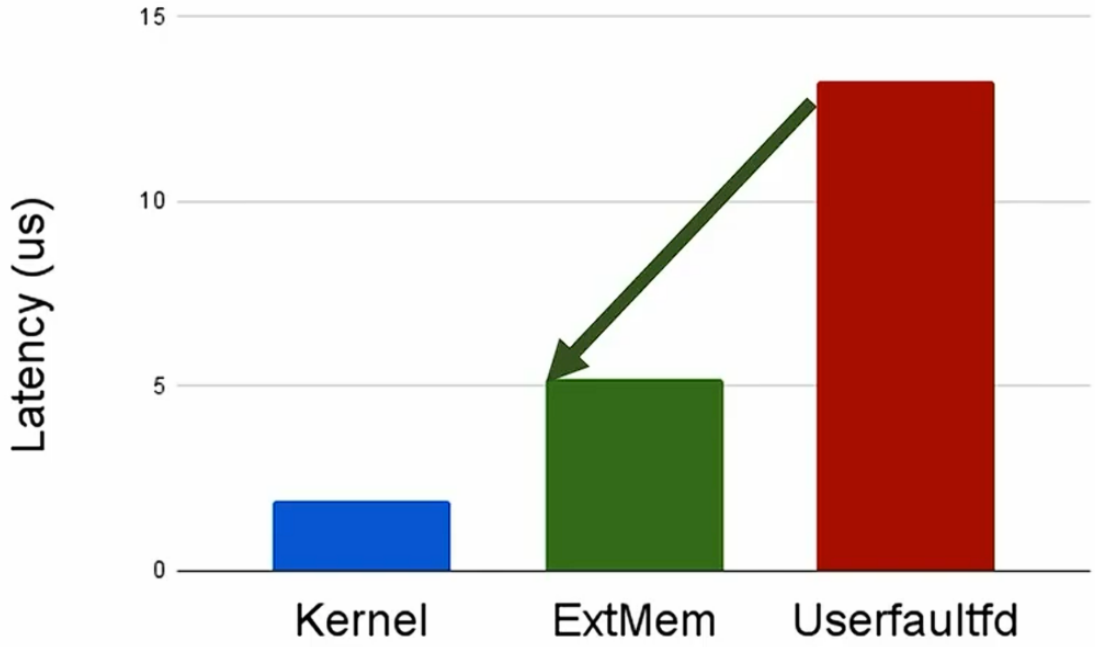
Intel Xeon 5218, 32 cores

198 GB DDR4 DRAM

NVMe 2.7 GB/s SSD

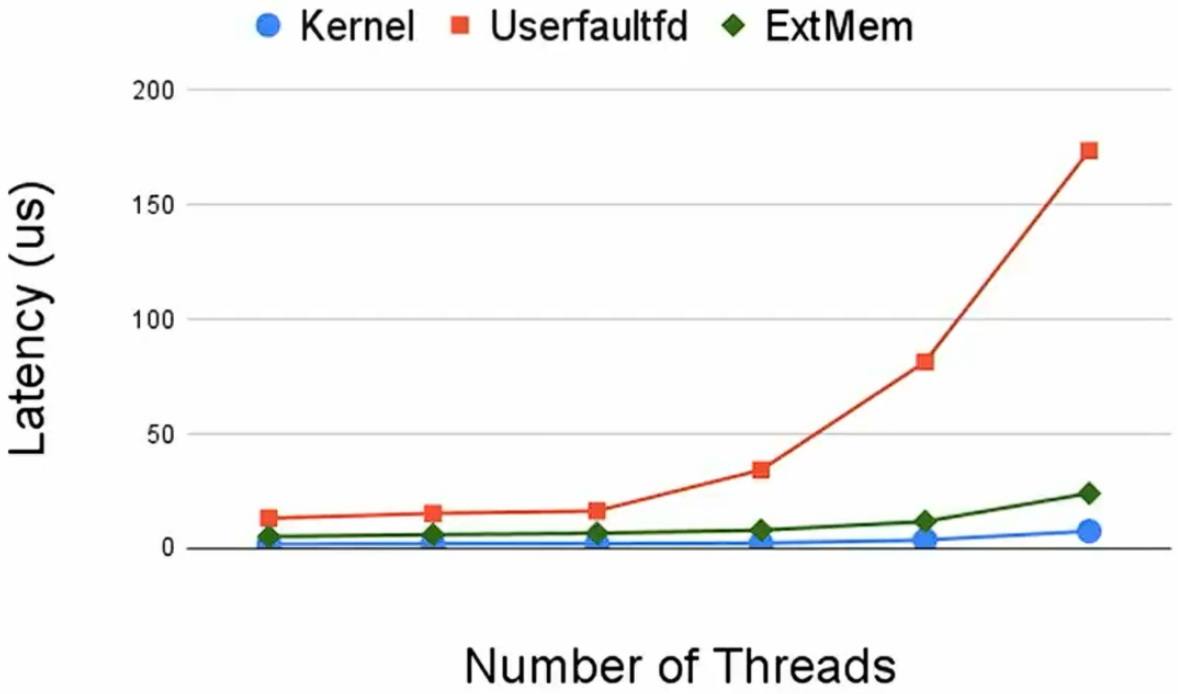
Upcall minor page fault latency

ExtMem's upcall has better latency than userfaultfd by eliminating expensive IPC's



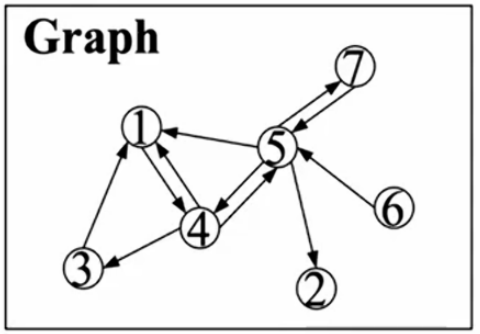
Scalability with concurrent faulting threads

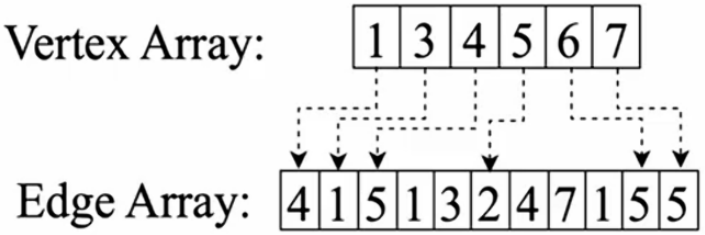
ExtMem's upcall has better scalability than userfaultfd by eliminating serialization



Improving Page rank performance

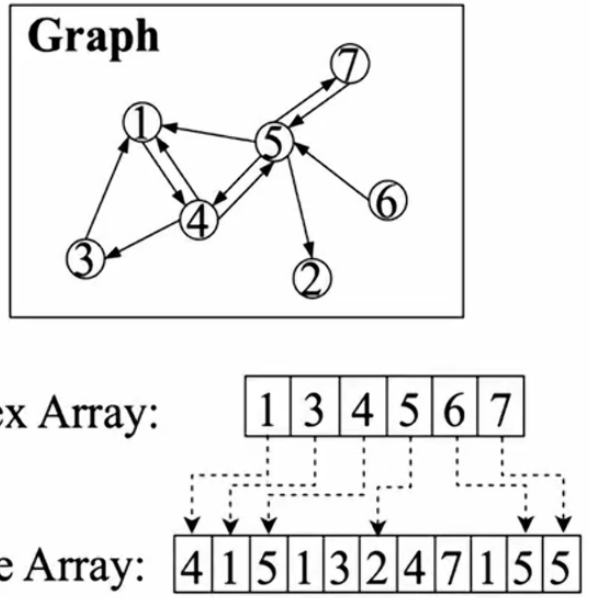
- A large scale graph processing: PageRank
- Intution: Vertex array stays hot and edge array is cold after being used once.
- Custom Policies:
  - Pin vertex array
  - Discard + prefetch window on edge array





Page Rank System setup

- Running GAP benchmark suite
- twitter dataset
- 50% (~7GB) fitting in memory
- 4 threads
- Runtime of 10 iterations measured



SYSTOPIA

Case Study: Page Rank

ExtMem's special policy (ExtMem-PR) has better performance than the Linux default policies